GUION COMPLETO — Charla Meta-Rock

Nerdearla Chile 2026 · Viernes 17 de abril · 12:00 hrs

25 min + 5 min Q&A · 23 slides

SLIDE 1 — Cover (0:00 — 0:15)

Contenido: "Construyendo un Asistente IA Multimodal en Meta Glasses" / Fernando Silva T / AWS Community Builder

🎤 **Guion para principiantes**

"Hola a todos, bienvenidos. Soy Fernando Silva, desarrollador de Chile y AWS Community Builder. Hoy les voy a mostrar algo que construí y que me tiene bastante emocionado."

🔧 **Info técnica**

- Charla nivel 100, no asumir conocimientos previos de AWS
- 15 segundos máximo en este slide

SLIDE 2 — Sobre mí (0:15 — 0:30)

Contenido: Dos columnas. Izquierda: datos personales. Derecha: "Construyo aplicaciones de IA y arquitecturas con servicios cloud"

🎤 **Guion para principiantes**

"Rápidamente: soy desarrollador y AWS Community Builder desde Chile. Me dedico a construir aplicaciones de inteligencia artificial usando servicios en la nube. Y hoy les traigo un proyecto que combina lentes inteligentes con IA."

🔧 **Info técnica**

- AWS Community Builder: programa de AWS que reconoce a desarrolladores que comparten conocimiento
- No extenderse, 15 segundos

SLIDE 3 — Agenda (0:30 — 0:50)

Contenido: 4 bloques: Contexto + Desafíos técnicos + Demo grabada + Lecciones

🎤 **Guion para principiantes**

"El plan es simple. Primero les explico las piezas: qué son estos lentes y qué es Amazon Bedrock. Después vemos cuatro desafíos técnicos que tuve que resolver, con código real. Les muestro una demo grabada del proyecto funcionando. Y cerramos con lecciones. Vamos."

🔧 **Info técnica**

- 4 bloques en vez de 5 (se quitó demo inicial separada)
- Timing: Contexto ~5min, Desafíos ~10min, Demo ~3min, Cierre ~3min

---

SLIDE 4 — Hook: F.R.I.D.A.Y. (0:50 — 1:30)

Contenido: "¿Se acuerdan de F.R.I.D.A.Y., la asistente de Tony Stark? Yo la construí, se llama Viernes"

🎤 **Guion para principiantes**

(Pausa. Mirar a la audiencia.)

"¿Cuántos acá han visto Iron Man?"

(Esperar manos.)

"¿Se acuerdan de F.R.I.D.A.Y., la asistente de Tony Stark? La que le decía qué tenía al frente, le respondía preguntas, todo a través de sus lentes..."

(Pausa dramática.)

"Bueno... yo la construí. Y se llama Viernes."

(Si tienes los lentes: ponértelos en este momento.)

🔧 **Info técnica**

- F.R.I.D.A.Y. = Female Replacement Intelligent Digital Assistant Youth, asistente de Iron Man después de Jarvis
- El nombre "Viernes" es la traducción literal, funciona como wake word en español
- Este es el momento de conexión emocional más importante de la charla

---

SLIDE 5 — Divider: Las piezas del rompecabezas (1:30 — 1:40)

Contenido: "Meta Glasses · Amazon Bedrock · IA Multimodal"

🎤 **Guion para principiantes**

"Para entender cómo funciona, necesitamos hablar de tres cosas: los lentes, un servicio de Amazon llamado Bedrock, y qué significa inteligencia artificial multimodal."

🔧 **Info técnica**

- Transición de 10 segundos, no demorarse

SLIDE 6 — Meta Smart Glasses (1:40 — 2:40)

Contenido: Imagen de los lentes + Cámara 12MP, Micrófono, Parlantes, Bluetooth, SDK DAT

🎤 **Guion para principiantes**

"Primero: los Meta Smart Glasses. Se ven como lentes de sol normales, pero tienen una computadora adentro."

(Señalar la imagen o mostrar los lentes físicos)

"Tienen una cámara de 12 megapíxeles — como la de un teléfono. Tienen micrófono para escucharte. Tienen parlantes para hablarte. Y se conectan a tu teléfono por Bluetooth, como unos audífonos."

"Lo interesante es que Meta, la empresa detrás de Facebook, liberó un kit de herramientas para desarrolladores. Se llama DAT SDK. Eso nos permite crear nuestras propias apps que acceden a la cámara de los lentes."

🔧 **Info técnica**

- Meta Wearables DAT SDK v0.4.0 (Device Access Toolkit)
- Requiere Meta AI app v254+ instalada y lentes emparejados
- El SDK entrega video en formato I420 (YUV planar) a 720p/30fps
- Conexión BLE para control + BTC para datos de video
- Firmware v20+ requerido en los lentes
- Android 12+ (API 31+)

SLIDE 7 — ¿Qué es Amazon Bedrock? (2:40 — 4:00)

Contenido: Imagen con diagrama de modelos + "El menú de modelos de IA" + bullets

🎤 **Guion para principiantes**

"Segundo: Amazon Bedrock. Les voy a explicar con una analogía."

"Imaginen que van a un restaurant. El restaurant tiene un menú con distintos platos de distintos chefs. Ustedes eligen qué quieren, lo piden, y el restaurant se encarga de cocinarlo y servirlo."

(Señalar el diagrama)

"Bedrock es exactamente eso, pero con inteligencia artificial. Es un servicio de Amazon donde tú eliges qué modelo de IA quieres usar. Hay modelos de Anthropic, de Meta, de Amazon, de Mistral... cada uno con sus fortalezas."

"Tú no necesitas instalar nada. No necesitas tener computadores potentes. No necesitas ser experto en inteligencia artificial. Solo eliges del menú, haces una llamada, y pagas por lo que usas."

"Yo elegí Claude Sonnet 4.5 de Anthropic, porque es muy bueno analizando imágenes. Y lo mejor: si mañana quiero cambiar a otro modelo, cambio UNA línea de código. El resto queda igual."

🔧 **Info técnica**

- Amazon Bedrock: servicio fully managed de AWS para modelos fundacionales
- Converse API: API unificada que abstrae las diferencias entre modelos
- Modelo usado: `us.anthropic.claude-sonnet-4-5-20250929-v1:0` (cross-region inference)
- Pricing: ~$3/M input tokens, ~$15/M output tokens para Sonnet 4.5
- Una imagen de 30KB ≈ ~1000 tokens ≈ ~$0.003 por request
- No requiere provisionar endpoints ni gestionar infraestructura
- AWS SDK for Kotlin `bedrockruntime` v1.6.12

SLIDE 8 — ¿Qué es IA Multimodal? (4:00 — 4:40)

Contenido: Imagen diagrama + "Texto + Imagen → Respuesta"

🎤 **Guion para principiantes**

"Tercero: IA multimodal. La mayoría de los chatbots que ustedes conocen — ChatGPT, por ejemplo — funcionan con texto. Tú escribes algo y te responden con texto."

"Pero un modelo multimodal puede entender texto E imágenes al mismo tiempo. Es como la diferencia entre hablar por teléfono y hacer una videollamada. En la videollamada la otra persona puede VER lo que tú ves."

"Eso es lo que hace posible que Viernes 'vea' a través de los lentes. Le mando la foto de lo que estoy viendo junto con mi pregunta, y la IA analiza las dos cosas juntas."

🔧 **Info técnica**

- Multimodal = múltiples modalidades de input (texto, imagen, audio, video)
- Claude Sonnet 4.5 soporta imagen + texto en un solo request via ContentBlock
- La imagen se envía como JPEG en base64 dentro de `ContentBlock.Image(ImageBlock)`
- El modelo decide internamente si la imagen es relevante para la respuesta
- Max tokens de respuesta configurado en 300 para mantener respuestas concisas

---

SLIDE 9 — Arquitectura (4:40 — 5:30)

Contenido: Imagen diagrama 4 pasos + "Lentes → App → Bedrock → Voz"

🎤 **Guion para principiantes**

"Y la arquitectura — cómo está armado todo — es así de simple. Son cuatro pasos."

(Señalar cada caja en el diagrama)

"Uno: los lentes capturan lo que estoy viendo y lo mandan a mi teléfono por Bluetooth. Dos: la app en el teléfono toma una foto de lo que ven los lentes y la junta con mi pregunta. Tres: manda todo a Amazon Bedrock, donde Claude lo analiza. Y cuatro: la respuesta vuelve al teléfono y se lee en voz alta."

"No hay nada más. No hay servidores intermedios, no hay infraestructura compleja. La app habla directo con Bedrock."

🔧 **Info técnica**

- Arquitectura directa: Android → Bedrock (sin API Gateway, Lambda, ni backend)
- AWS SDK for Kotlin se ejecuta directamente en el dispositivo Android
- Credenciales via BuildConfig (inyectadas desde local.properties en build time)

- Comunicación HTTPS directa al endpoint de Bedrock
- Latencia end-to-end: 2-3 segundos (bottleneck es la inferencia en Bedrock)

SLIDE 10 — Divider: Cuatro desafíos técnicos (5:30 — 5:40)

Contenido: "Código · Decisiones · Resultados"

🎤 **Guion para principiantes**

"Ok, suena simple cuando lo explico así. Pero construirlo tuvo desafíos reales. Les voy a mostrar los cuatro problemas más interesantes que tuve que resolver, con el código real."

🔧 **Info técnica**

- Cada desafío sigue el patrón: Problema → Decisión → Código → Resultado
- Los 4 desafíos cubren: voz, video, IA, y orquestación

SLIDE 11 — Desafío 1: Wake Word gratis (5:40 — 7:00)

Contenido: Imagen flujo + "Oye Viernes sin pagar un peso" + SpeechRecognizer con resultados parciales

🎤 **Guion para principiantes**

"Primer desafío: necesitaba que la app estuviera escuchando todo el tiempo, esperando que yo dijera 'Oye Viernes'. Como cuando le dices 'Hey Siri' al iPhone o 'Ok Google' al teléfono."

"El problema es que las herramientas profesionales para hacer esto cuestan plata. Hay empresas que cobran por detectar palabras de activación."

"Mi solución: resulta que Android ya tiene reconocimiento de voz gratis incluido. El truco está en pedirle que me mande resultados parciales. ¿Qué significa eso? Que mientras tú hablas, Android va mandando lo que CREE que estás diciendo, antes de que termines la frase."

"Entonces si yo digo 'Oye Viernes, ¿qué hora es?', Android primero manda 'oye', después 'oye vier...', después 'oye viernes'. Y cuando mi app detecta 'oye viernes' en esos resultados parciales... ¡se activa!"

🔧 **Info técnica**

- Android `SpeechRecognizer` con `EXTRA_PARTIAL_RESULTS = true`
- Wake words: "oye viernes", "hey friday", "oye biernes", "hey viernes"
- Variantes incluidas por errores comunes del reconocedor (V→B)
- Modo pasivo: escucha continua, solo revisa parciales
- Modo activo: post-wake word, captura transcript completo
- Error 7 (`ERROR_NO_MATCH`): se reinicia automáticamente
- Latencia de detección: ~500ms
- Trade-off: mayor consumo de batería (~15%/hora) vs cero costo de SDK

SLIDE 12 — Código: Wake Word (7:00 — 8:30)

Contenido: Imagen con screenshot de código + "Sin SDKs. Sin costos. Android nativo."

🎤 **Guion para principiantes**

(Ir lento, dar tiempo para leer)

"Y el código para hacer esto es sorprendentemente corto. Miren:"

"Primero le digo a Android: 'mándame resultados parciales'. Esa es la línea clave, el truco de todo esto."

"Después, cada vez que Android me manda un resultado parcial, lo reviso: ¿contiene 'oye viernes'? Si sí, activo al asistente."

"Son básicamente 5 líneas de código. Sin instalar nada extra, sin pagar licencias, sin servidores. Solo usando lo que Android ya trae."

"Y un detalle que me gusta: incluí la variante 'oye biernes' con B, porque el reconocedor de voz a veces confunde la V con la B. Siempre piensen en cómo puede fallar el usuario real."

🔧 **Info técnica**

```
 // Modo pasivo
putExtra(RecognizerIntent.EXTRA_PARTIAL_RESULTS, true)

// En onPartialResults:
val text = partialResults?.getStringArrayList(
    SpeechRecognizer.RESULTS_RECOGNITION
)?.firstOrNull()?.lowercase()
if (WAKE_WORDS.any { text.contains(it) }) {
```

```
    onWakeWord()
}
```

- `RecognitionListener.onPartialResults()` se llama múltiples veces durante el habla
- `onResults()` se usa en modo activo para capturar el transcript completo
- El recognizer se destruye y recrea entre sesiones para evitar memory leaks
- Handler con delay de 300ms entre restart para que el recognizer anterior se libere

---

SLIDE 13 — Desafío 2: Video → Imagen (8:30 — 9:30)

Contenido: Imagen pipeline + "De ~2MB a ~30KB"

🎤 Guion para principiantes

"Segundo desafío: los lentes graban video todo el tiempo, ¿cierto? Pero yo no necesito mandarle un video completo a la IA. Sería como mandarle a un amigo un video de 10 minutos cuando le puedes mandar una sola foto y preguntarle '¿qué ves acá?'."

"Eso es exactamente lo que hago. En el momento que tú preguntas, tomo UNA sola foto del video."

"El problema es que esa foto sale en un formato que la IA no entiende — es como si estuviera en un idioma raro. Entonces la convierto a JPEG, que es el formato de foto que todos conocemos."

"Y antes de enviarla, la achico. No necesito una foto de 10 megapíxeles para que la IA entienda qué hay en la imagen. La reduzco a algo bien liviano, como 30 kilobytes. Es como comprimir una maleta grande en una mochila."

"Resultado: en vez de mandar un video pesado de 2 megabytes, mando una fotito de 30 kilobytes. Más rápido, más barato, y la IA entiende igual."

🔧 Info técnica

- Meta DAT SDK entrega frames en I420 (YUV planar): Y plane + U plane + V plane
- Pipeline: I420 → NV21 (intercalar U/V) → YuvImage → JPEG 85% (pantalla) → Bitmap.createScaledBitmap(512px) → JPEG 60% (Bedrock)
- `convertI420toNV21()` : copia Y tal cual, intercala V y U byte a byte
- 85% JPEG para mostrar en la app (buena calidad visual)

- 60% JPEG + 512px max width para enviar a Bedrock (optimizar tokens y latencia)
- Frame se captura via lambda `currentFrameProvider` en el momento del transcript

---

SLIDE 14 — Código: Pipeline de imagen (9:30 — 10:30)

Contenido: Imagen con screenshot de código + "85% pantalla / 60% Bedrock"

🎤 **Guion para principiantes**

"El código tiene tres pasos. Primero convierto el formato del video a algo que Android entienda. Después lo comprimo como foto al 85% de calidad — eso es para que se vea bien en la pantalla de la app."

"Y el tercer paso es el importante: achico la foto y la comprimo más, al 60%, para mandarla a Bedrock. ¿Por qué dos compresiones distintas? Porque tienen propósitos distintos. Una es para que tú veas buena calidad, la otra es para que sea liviana y rápida de enviar."

🔧 **Info técnica**

```
// 1. I420 → NV21
val nv21 = convertI420toNV21(byteArray, width, height)
// 2. NV21 → JPEG 85% (display)
YuvImage(nv21, ImageFormat.NV21, w, h, null)
    .compressToJpeg(Rect(0,0,w,h), 85, stream)
// 3. Scale + JPEG 60% (Bedrock)
val scaled = Bitmap.createScaledBitmap(bitmap, 512, ...)
scaled.compress(Bitmap.CompressFormat.JPEG, 60, stream)
```

SLIDE 15 — Desafío 3: Hablando con Bedrock (10:30 — 13:00)

Contenido: Imagen con screenshot de código + "Esto es todo. Bedrock hace el resto." ⭐ **SLIDE ESTRELLA — IR MUY LENTO**

🎤 **Guion para principiantes**

"Tercer desafío, y el más importante: ya tengo la foto y la pregunta del usuario. ¿Cómo se lo mando a la inteligencia artificial?"

"Bedrock tiene algo que se llama la Converse API. Piénsenla como un WhatsApp con la IA. Tú le mandas un mensaje que puede tener texto, imágenes, y unas instrucciones de personalidad."

(Señalar el código línea por línea, MUY lento)

"Miren el código. Primero elijo el modelo: Claude Sonnet 4.5. Si mañana quiero usar otro modelo, cambio solo esta línea."

"Después le doy la personalidad. Le digo: 'Eres Viernes, un asistente inspirado en F.R.I.D.A.Y. de Iron Man. Responde en máximo 2-3 oraciones.' Eso es lo que hace que Viernes sea Viernes y no un chatbot genérico."

"Y finalmente le mando el contenido: la imagen que capturaron los lentes y la pregunta del usuario. Texto e imagen juntos, en un solo mensaje."

(Pausa larga)

"Esto es TODO el código para hablar con una de las inteligencias artificiales más avanzadas del mundo. No tuve que entrenar ningún modelo. No necesité computadores potentes. No necesité ser experto en machine learning. Bedrock hace el resto."

🔧 **Info técnica**

```
val request = ConverseRequest {
    modelId = "us.anthropic.claude-sonnet-4-5-20250929-v1:0"
    system = listOf(SystemContentBlock.Text(systemPrompt))
    messages = listOf(
        Message {
            role = ConversationRole.User
            content = listOf(
                ContentBlock.Image(ImageBlock {
                    format = ImageFormat.Jpeg
                    source = ImageSource.Bytes(imageBytes)
                }),
                ContentBlock.Text(prompt)
            )
        }
    )
    inferenceConfig {
        maxTokens = 300
        temperature = 0.7F
    }
}
```

```
val response = client.converse(request)
    .output!!.asMessage().content.first().asText()
```

- `ConverseRequest`: API unificada de Bedrock, funciona igual para cualquier modelo
- `SystemContentBlock.Text`: define personalidad y comportamiento (system prompt)
- `ContentBlock.Image` + `ContentBlock.Text`: input multimodal en un solo request
- `maxTokens = 300`: limita respuesta a ~2-3 oraciones
- `temperature = 0.7`: balance entre creatividad y coherencia
- Cross-region inference (`us.` prefix): permite usar el modelo desde cualquier región
- El system prompt cambia según idioma (español/inglés) e incluye fecha/hora actual
- Sin historial de conversación: cada request es stateless (decisión de diseño)

SLIDE 16 — Desafío 4: Orquestando todo (13:00 — 14:00)

Contenido: Imagen máquina de estados + "5 estados claros"

🎤 **Guion para principiantes**

"Último desafío: tengo cuatro sistemas funcionando — el reconocimiento de voz, la cámara de los lentes, la inteligencia artificial, y la voz que lee las respuestas. ¿Cómo hago para que no se pisen entre sí?"

"La solución es algo que en programación se llama máquina de estados. Suena complicado pero es simple: la app siempre está en UNO de cinco estados posibles."

(Señalar cada estado en el diagrama)

"IDLE: la app está tranquila, esperando que digas 'Oye Viernes'. LISTENING: te está escuchando, esperando tu pregunta. PROCESSING: mandó todo a Bedrock y está esperando la respuesta. RESPONDING: está leyendo la respuesta en voz alta. Y después vuelve a IDLE."

"El usuario ve en qué estado está la app en todo momento, con un indicador de colores en la pantalla."

🔧 **Info técnica**

- `NovaState` enum: IDLE, LISTENING, PROCESSING, RESPONDING, ERROR

- `NovaUiState` data class con: state, transcript, response, sentWithImage, error
- StateFlow + collectAsStateWithLifecycle para UI reactiva en Compose
- El overlay (`NovaOverlay.kt`) muestra el estado con animaciones y colores
- ERROR → automáticamente vuelve a IDLE con passive listening

---

SLIDE 17 — Código: Estados (14:00 — 15:30)

Contenido: Imagen con código del enum + dos modos de activación

🎤 Guion para principiantes

"El código es un enum — una lista de opciones posibles. La app solo puede estar en una de estas cinco opciones a la vez."

"Y hay un detalle de diseño que me gusta mucho: hay dos formas de activar a Viernes."

"Si lo activaste por voz, diciendo 'Oye Viernes', después de responderte vuelve a dormir. Tienes que decir 'Oye Viernes' de nuevo para la siguiente pregunta. Es como Siri."

"Pero si lo activaste tocando el botón del micrófono, después de responderte sigue escuchando. Puedes hacer varias preguntas seguidas sin repetir 'Oye Viernes'. Es como una conversación."

"Esto es exactamente lo que van a ver en la demo."

🔧 Info técnica

```
enum class NovaState { IDLE, LISTENING, PROCESSING, RESPONDING, ERROR }
```

- Flag `activatedByVoice` controla el comportamiento post-respuesta
- Voz (`activatedByVoice = true`): TTS onDone → IDLE + startPassiveListening
- Botón (`activatedByVoice = false`): TTS onDone → LISTENING + startActiveListening
- `processingJob` es un coroutine Job cancelable (dismiss cancela el request a Bedrock)
- Todo corre en `viewModelScope` con `Dispatchers.IO` para el request a Bedrock

SLIDE 18 — Divider: Demo (15:30 — 15:40)

Contenido: "Demo — Video del flujo completo 🎬"

🎤 **Guion para principiantes**

"Ok. Ahora que saben cómo funciona por dentro, les voy a mostrar todo funcionando junto."

🔧 **Info técnica**

- Transición de 10 segundos

SLIDE 19 — Demo grabada (15:40 — 19:00)

Contenido: 6 pasos + video embebido + "Esto no es un concepto. Esto funciona hoy."

🎤 **Guion para principiantes**

"Ahora ustedes saben qué pasa por dentro. Así que mientras ven el video, fíjense en cada paso."

(Reproducir video)

(Narrar mientras se reproduce:) "Ahí está en IDLE, esperando... ahora digo 'Oye Viernes' — miren, el indicador cambió a naranja, está escuchando... hago la pregunta... capturó la foto de los lentes... ahora está en amarillo, procesando, Bedrock está analizando la imagen y el texto... y ahí está la respuesta, en verde, el teléfono la lee en voz alta..."

"Y vuelve a IDLE. Listo para la siguiente pregunta."

(Después del video:) "Esto no es un concepto. Esto no es un mockup. Esto funciona hoy, con servicios que cualquiera de ustedes puede usar."

🔧 **Info técnica**

- Video debe mostrar: wake word → overlay LISTENING → pregunta → overlay PROCESSING → respuesta → overlay RESPONDING → TTS → vuelta a IDLE
- Idealmente mostrar también el modo botón (conversación continua)
- Duración ideal del video: 2-3 minutos

SLIDE 20 — Divider: Lecciones (19:00 — 19:10)

Contenido: "Tres takeaways para llevarse a casa"

🎤 **Guion para principiantes**

"Tres cosas que aprendí construyendo esto."

SLIDE 21 — Tres lecciones (19:10 — 21:00)

Contenido: 3 bloques: No reinventes la rueda / Optimiza / Piensa en tu usuario

🎤 **Guion para principiantes**

"Uno: no reinventen la rueda. La detección de voz la resolví con algo que Android ya trae gratis. La inteligencia artificial con Bedrock. La voz que lee las respuestas, también con Android. El 90% de este proyecto es juntar piezas que ya existen. Tu trabajo como desarrollador es ser buen pegamento."

"Dos: optimicen para el caso real. Yo no mando un video completo a la IA. Mando una sola foto de 30 kilobytes. No pido respuestas de 2000 palabras. Pido 2-3 oraciones. Cada decisión de optimización hace que sea más rápido y más barato."

"Tres: piensen en su usuario desde el día uno. Hice la app bilingüe desde el inicio — español e inglés. Incluí variantes de pronunciación del wake word porque la gente no habla perfecto. Agregué dos modos de activación para distintos contextos. Son pequeños detalles que hacen la diferencia entre un demo y un producto."

🔧 **Info técnica**

- Latencia end-to-end: 2-3 segundos
- Costo por interacción: ~$0.01-0.02 USD
- Consumo batería: ~15%/hora
- APK size: ~12MB (sin contar dependencias del SDK de Meta)
- Precisión wake word: ~95% en ambientes silenciosos

SLIDE 22 — Recursos (21:00 — 22:00)

Contenido: Dos columnas. Repo + web + Bedrock | Links de docs + hashtags

🎤 **Guion para principiantes**

"Todo el código es open source. El repositorio en GitHub tiene instrucciones paso a paso para que lo repliquen. Bedrock tiene un tier gratuito para empezar a experimentar."

"Y si quieren hacer algo parecido pero no tienen los lentes: no los necesitan. Pueden usar la cámara del teléfono. El flujo es exactamente el mismo. Los lentes son el factor wow, pero la arquitectura funciona con cualquier cámara."

🔧 **Info técnica**

- Repo: github.com/fernandosilvot/aws-bedrock-meta-glasses
- Blog: dev.to/aws-espanol/crea-tu-asistente-personal-en-los-meta-glasses-4nen
- Bedrock docs: docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference.html
- Meta DAT SDK: wearables.developer.meta.com/docs/reference/android/dat/0.4

SLIDE 23 — ¡Gracias! (22:00 — 22:30)

Contenido: "¡Gracias! 🙌" + contacto

🎤 **Guion para principiantes**

"Eso es todo. Muchas gracias por su tiempo. Si quieren ver los lentes de cerca, probar la app, o tienen preguntas, estoy acá. ¡Gracias!"

(Aplausos → Q&A 5 minutos)

RESPUESTAS PARA Q&A

"¿Cuánto cuesta usar Bedrock?" Para principiantes: "Cada vez que le hago una pregunta con foto cuesta como 1 o 2 centavos de dólar. Si hago 100 preguntas al día, son 1 a 2 dólares al mes. Y hay un tier gratuito para empezar." Técnico: Input ~$3/M tokens, output ~$15/M tokens. Una imagen de 30KB ≈ 1000 tokens ≈ $0.003. Con maxTokens=300, cada request cuesta ~$0.01-0.02.

"¿Por qué no usar Meta AI?" Para principiantes: "Meta AI solo funciona en inglés y en algunos países. No puedo elegir qué modelo usa ni controlar mis datos. Con Bedrock yo elijo todo." Técnico: Meta AI no está disponible en LATAM, no expone API para customización, no permite elegir modelo ni ajustar system prompts.

"¿Se puede hacer sin los lentes?" Para principiantes: "Sí, pueden usar la cámara del teléfono. El flujo es el mismo: capturar foto, mandar a Bedrock, recibir respuesta." Técnico: Reemplazar `StreamViewModel.videoFrame` por `CameraX` capture. El `BedrockClient` y `NovaViewModel` quedan idénticos.

"¿Cuánto se demora en responder?" Para principiantes: "Entre 2 y 3 segundos desde que terminas de hablar hasta que escuchas la respuesta." Técnico: ~500ms wake word detection + ~200ms frame capture + ~100ms image compression + ~1500-2000ms Bedrock inference + ~200ms TTS init.

"¿Qué pasa si no hay internet?" Para principiantes: "No funciona sin internet. Toda la inteligencia está en la nube. Pero eso es intencional: mantiene la app liviana." Técnico: Toda la inferencia es server-side via Bedrock. No hay modelo on-device. Trade-off: latencia de red vs complejidad y tamaño del APK.